

SQL Functions – Character functions

Dr Mariana Rocha
School of Computer Science

CPMU 2007 Databases 1, Lecture 5

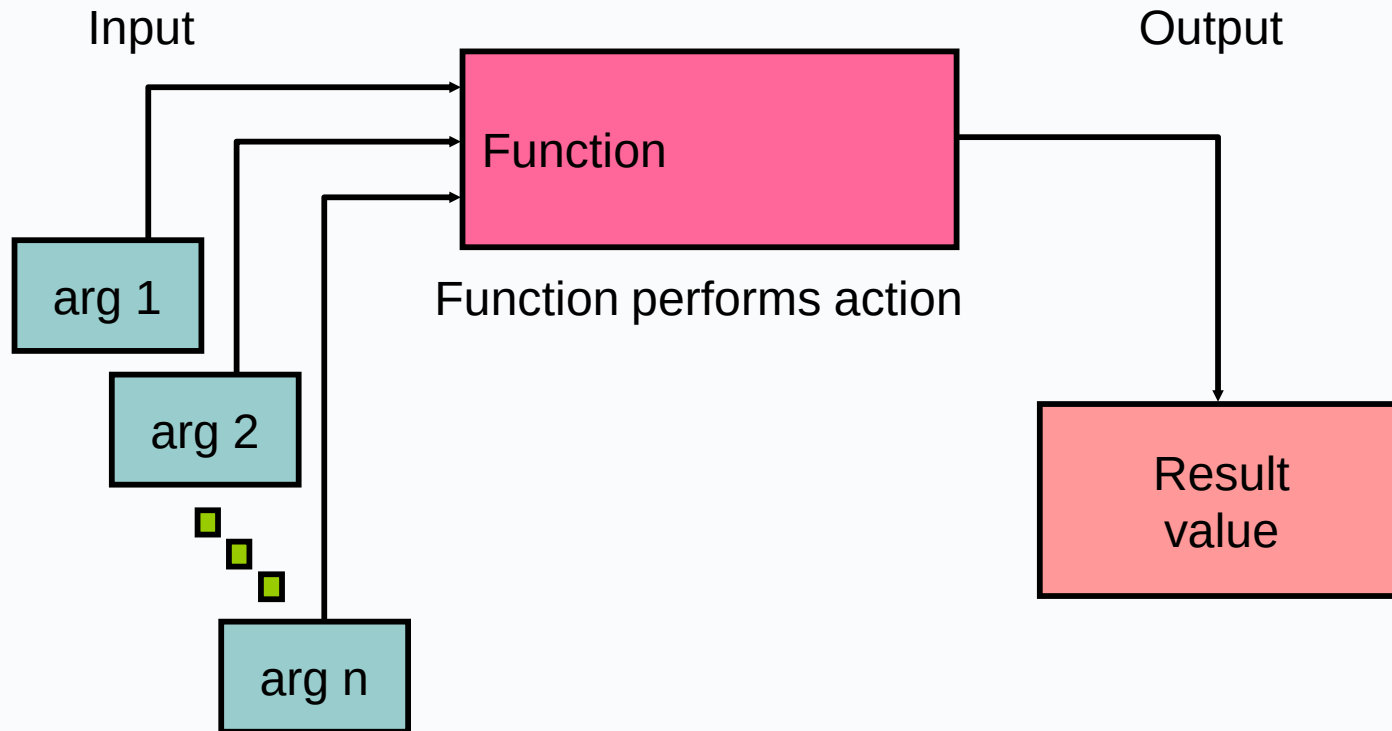
What are Functions in SQL

- So far, we have looked at how we can use SELECT queries to find specific values
- Functions are pre-built pieces of functionality that we can perform on our databases quickly and easily
- Functions can allow us to
 - Perform calculations on data
 - Modify individual data items
 - Manipulate output for groups of rows
 - Format dates and numbers for display
 - Convert column data types

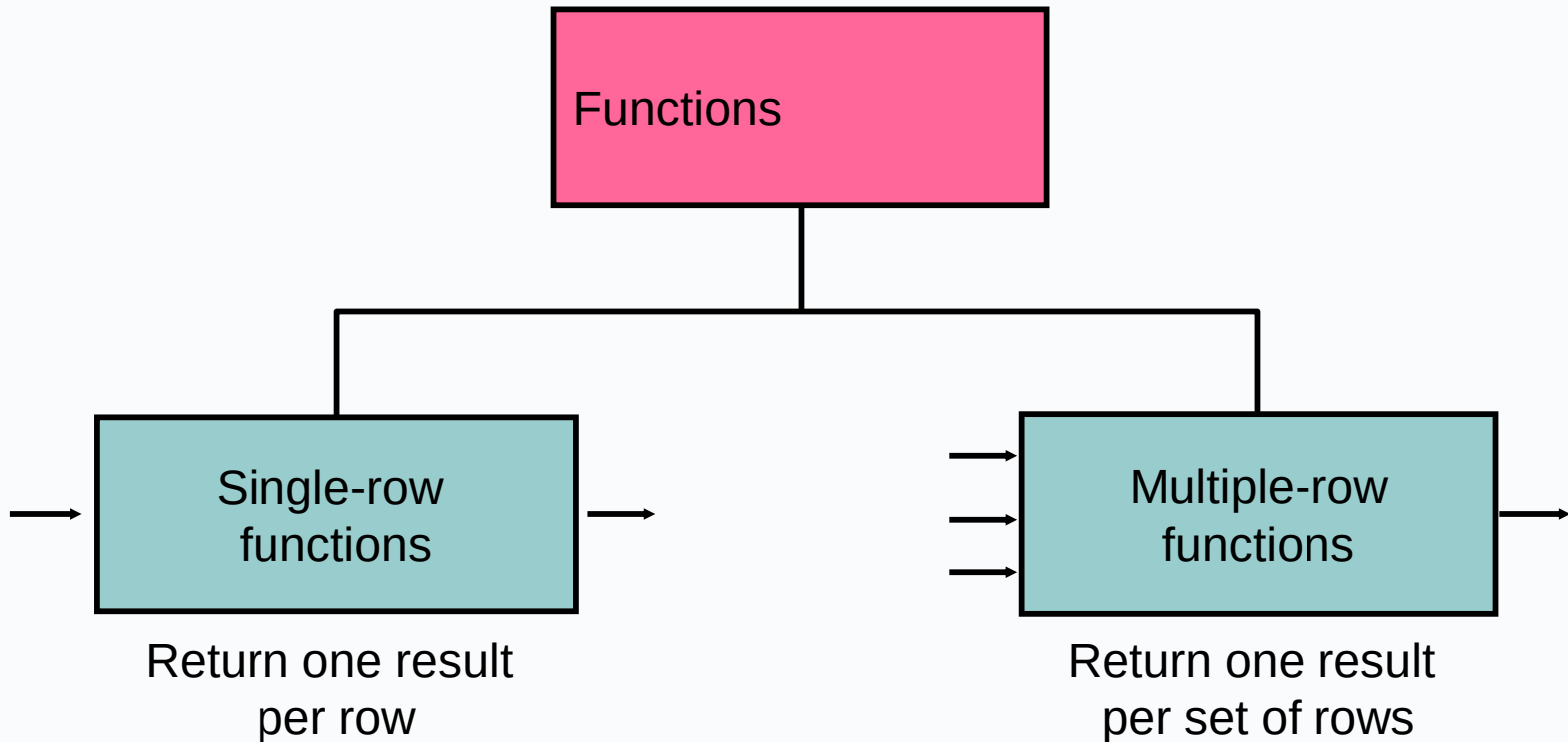
SQL Functions

- SQL functions **sometimes** take arguments and **always** return a value.
- SQL has a long list of built-in functions

SQL Functions



Types of SQL Functions



Single-row functions

- Single-row functions:
 - Manipulate data items
 - Access and works on each row and return a value per row
 - May return a value of a data type different to that referenced
 - Can be nested

```
function_name [(arg1, arg2, ...)]
```

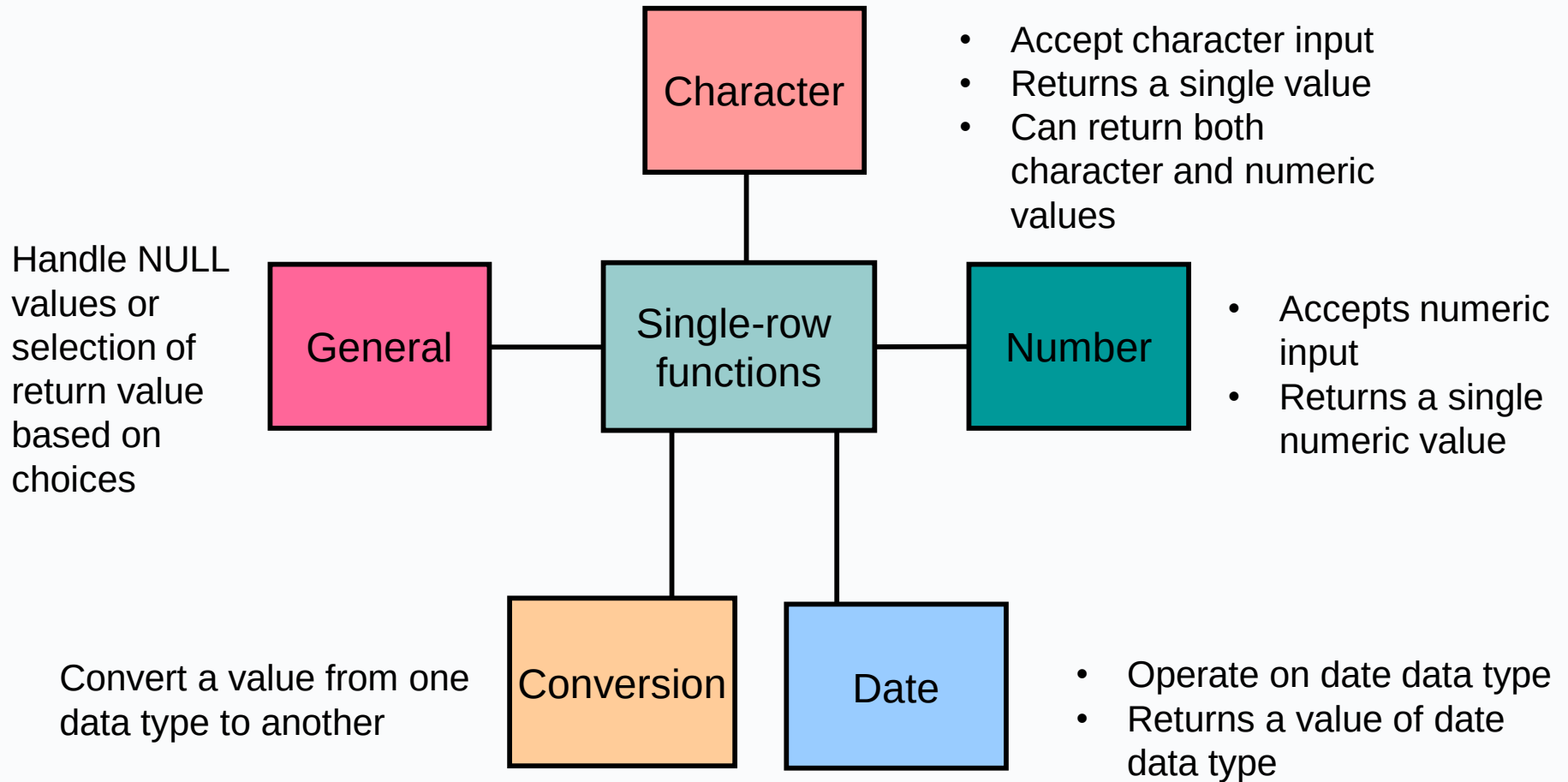
- Accept arguments that can be a column or an expression

Single-row functions

- An argument can be one of the following:
 - User-supplied constant
 - Column name
 - Expression
- Can be used in
 - Select
 - Where
 - Order By
 - Constraint

```
function_name [(arg1, arg2, ...)]
```

Single-row functions



Character functions

```
graph TD; A[Character functions] --> B[Case-manipulation functions]; A --> C[Character-manipulation functions]; B --- D[LOWER<br/>UPPER<br/>INITCAP]; C --- E[CONCAT<br/>SUBSTR<br/>LENGTH<br/>INSTR<br/>LPAD<br/>RPAD<br/>TRIM<br/>REPLACE];
```

Case-manipulation functions

LOWER
UPPER
INITCAP

Character-manipulation functions

CONCAT
SUBSTR
LENGTH
INSTR
LPAD
RPAD
TRIM
REPLACE

Character functions

Case-manipulation functions

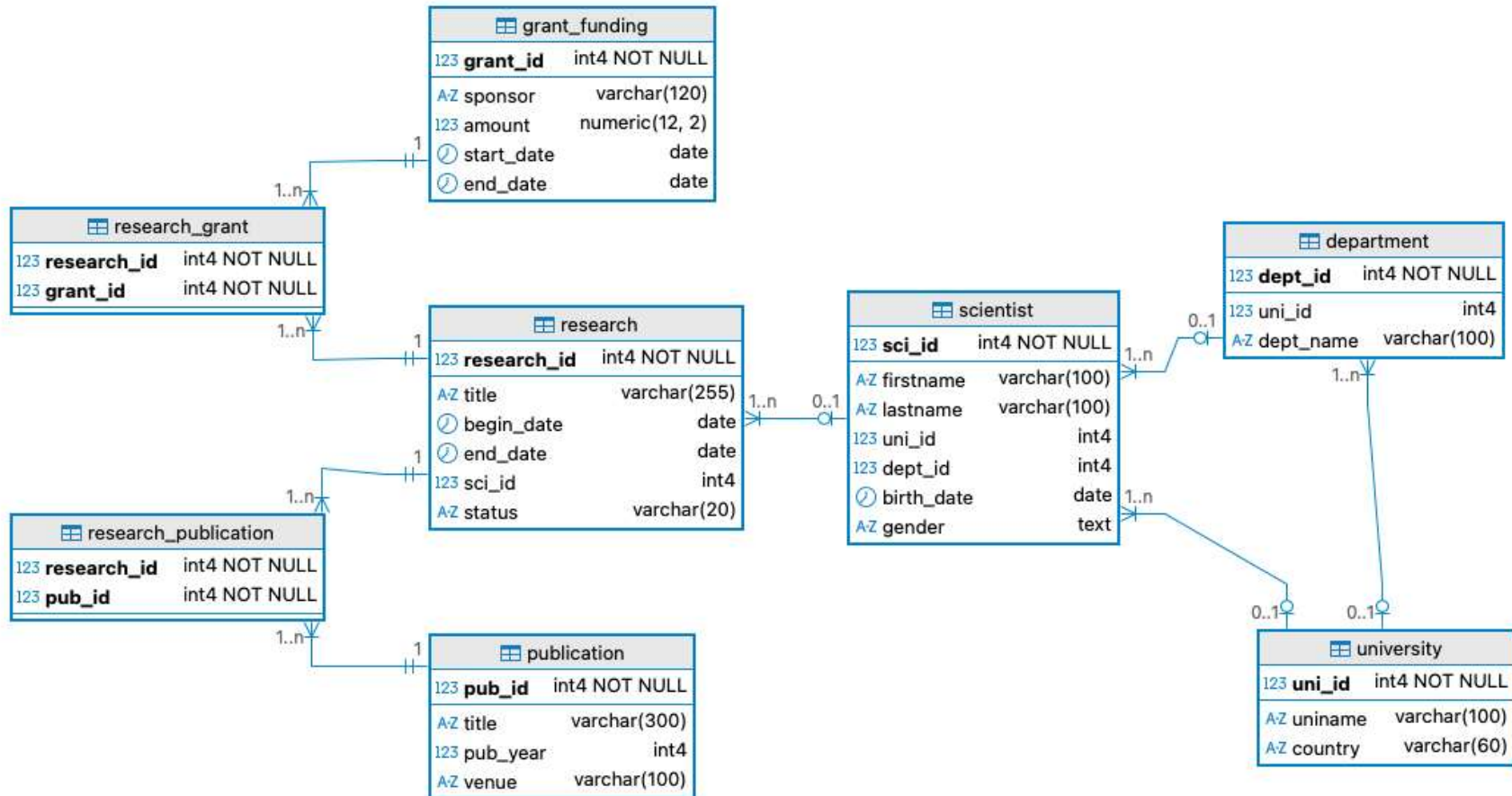
Clients	
clientID	Number(6)
Fullname	Varchar2(20)



clientId	Fullname
1	Joe Bloggs
2	Jim bloogs

Function	Result
<code>SELECT LOWER(Fullname) FROM Clients;</code>	joe bloggs jim bloogs
<code>SELECT UPPER(Fullname) FROM Clients;</code>	JOE BLOGGS JIM BLOOGS
<code>SELECT INITCAP(Fullname) FROM Clients;</code>	Joe Bloggs Jim Bloogs

We will look at examples using the following database



Case manipulation: upper() and lower()

Character functions can be used to convert strings to uppercase or lowercase using the UPPER() and LOWER() functions.

```
-- Uppercase research titles for display  
SELECT title, UPPER(title) "Upper Title"  
FROM research;
```

```
-- Lowercase venues for standardised export  
SELECT venue, LOWER(venue) "Lower Title"  
FROM publication;
```

Case manipulation: upper() and lower()

It can also be used to manipulate WHERE conditions.

```
-- Case-insensitive search for a scientist  
by last name  
SELECT firstname, lastname  
FROM scientist  
WHERE LOWER(lastname) = 'murphy';
```

Character functions

Character-manipulation functions

Function	Result	Objective
<code>CONCAT('Hello', 'World')</code>	HelloWorld	Concatenate
<code>SUBSTR('HelloWorld',1,5)</code>	Hello	Extract
<code>LENGTH('HelloWorld')</code>	10	Check length
<code>POSITION('W' IN 'Hello World')</code>	6	Find position of first occurrence
<code>LPAD('tech', 8, '0');</code>	0000tech	Pads the left
<code>RPAD('tech', 8, '0')</code>	tech0000	Pads the right
<code>REPLACE('JACK and JUE','J','BL')</code>	BLACK and BLUE	Replace character
<code>TRIM('H' FROM 'HelloWorld')</code>	elloWorld	

Case manipulation: concatenate()

It can be used to modify how the data is presented:

```
SELECT CONCAT(TRIM(firstname), ' ', TRIM(lastname), '  
(', TRIM(uniname), ')') " Scientist affiliation"  
FROM scientist  
JOIN university USING (uni_id);
```

Case manipulation: substring()

Extracts part of a string:

```
-- First 5 chars of university names (spot prefixes &  
duplicates)  
SELECT unname,  
SUBSTRING(unname FROM 1 FOR 10) "Uni prefix"  
FROM university;
```


Case manipulation: length()

The LENGTH function returns the length of a string. It can be used for data validation or to manage constraints.

```
-- Research title length
SELECT research_id, LENGTH(title) "Research
title length"
FROM research
ORDER BY title_len DESC;
```

Case manipulation: position()

Find the position of a substring within a string in the database you provided earlier

```
-- Find titles mentioning 'AI' anywhere
SELECT research_id, title,
POSITION('AI' IN UPPER(title)) "Position AI"
FROM research
WHERE POSITION('AI' IN UPPER(title)) > 0;
```

Case manipulation: lpad() and rpad()

- Pads a string on the left or right side with a specified character or characters until it reaches the desired length.
- These functions are mostly used for formatting the data in a certain way. The syntax is:

LPAD(string, length, fill_string) or LRPAD(string, length, fill_string)

Case manipulation: lpad() and rpad()

```
-- Zero-padded research codes (for reports)
SELECT research_id, 'RS-' || LPAD(CAST(research_id AS
VARCHAR), 5, '0') "Research code"
FROM research
ORDER BY 1;
```

Case manipulation: replace()

Used to replace occurrences of a substring within a string.

```
-- Normalise hyphenated last names to space-separated
SELECT sci_id,
       lastname "Last name",
       REPLACE(lastname, '-', ' ') "Normalised lastname"
FROM scientist
WHERE lastname LIKE '%-%';
```

Case manipulation: trim()

Removes leading and trailing spaces from a string. Trim function is mostly used for database cleanup and formatting.

```
-- Clean leading/trailing spaces from names
SELECT firstname "Raw first name", length(firstname)
"First name size",
TRIM(firstname) "Trimmed first name",
length(trim(firstname)) "First name size after trimming"
FROM scientist
WHERE firstname LIKE ' %' OR firstname LIKE '% ';
```

Adding a WHERE clause

-- Case-insensitive exact match for a project code prefix within the title

```
SELECT *  
FROM research  
WHERE LOWER(title) LIKE '%quantum study';
```

-- Case-sensitive (likely fewer or no matches, depending on data)

```
SELECT *  
FROM research  
WHERE title LIKE '%quantum study';
```

Another way of concatenating

- We can concatenate values using || or the function concat().
- Concat() can be a bit more efficient when we have many arguments.

```
-- Using ||  
SELECT 'Scientist ' || sci_id || ': ' || UPPER(lastname) ||  
' , ' || INITCAP(TRIM(firstname)) "Information"  
FROM scientist  
LIMIT 10;
```

```
-- Using CONCAT (two-arg chaining)  
SELECT CONCAT('Scientist ', sci_id, ': ', UPPER(lastname),  
' , ', INITCAP(TRIM(firstname))) "Information"  
FROM scientist  
LIMIT 10;
```

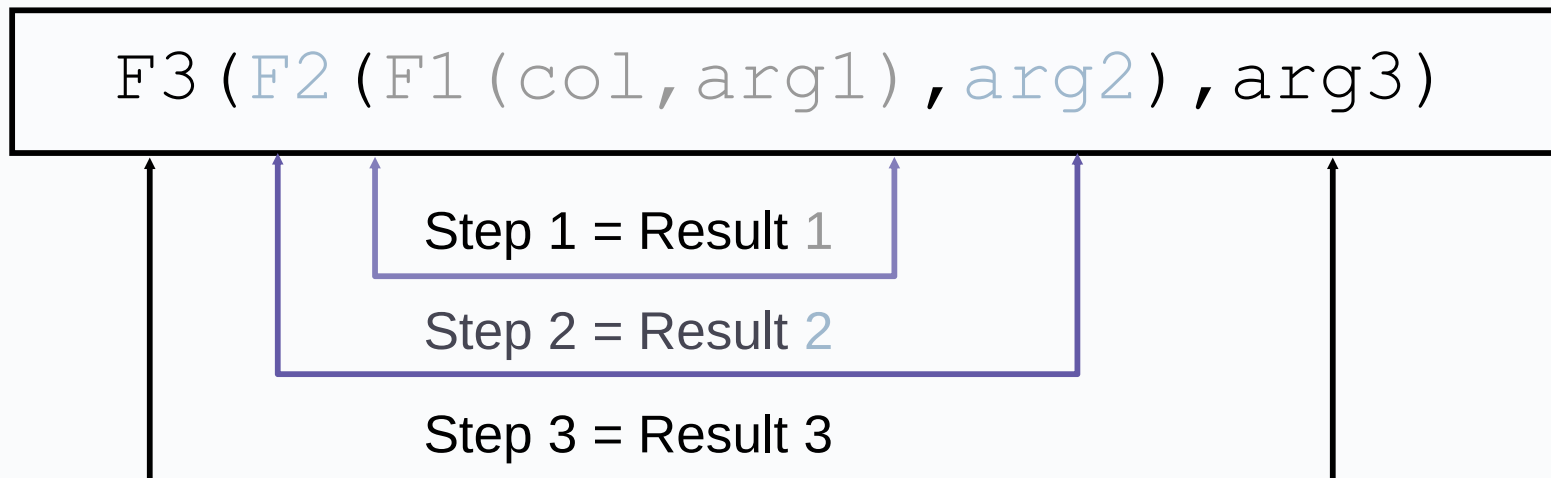

Another way of concatenating

- Be aware: when we use `||` to concatenate two values and one is NULL, the result is a NULL value. This way:
 - `'a' || NULL → NULL`
 - `CONCAT('a', NULL) → 'a'`
- As we usually prefer to keep any value that is not null, give preference to using `concat( )`
- You can also use `concat_ws( sep, arg)`, which will take a separator and arguments, skipping the NULL ones. Ws stands for “with separator”

```
SELECT 'a' || NULL;  
SELECT concat('a', NULL);  
SELECT concat_ws(' ', 'a', NULL, 'b');
```

Nesting functions

- Single-row functions can be nested
- Nested functions are evaluated from deepest level to the least deep level.



Nesting functions

```
--Nested functions
-- Lastname stub + country suffix, uppercased
SELECT lastname,
UPPER(CONCAT(SUBSTRING(REGEXP_REPLACE(lastname, '\s+',
'', 'g') FROM 1 FOR 8), '_IRL')) "Tag"
FROM scientist
JOIN university USING (uni_id)
WHERE university.country = 'Ireland'
LIMIT 15;
```

